

Table Matchings.

Say that we want to pair up the jobs (1,2,3) with the candidates (A,B,C); such a pairing is called a *matching*. The table below gives the ranked preferences of the jobs and candidates, with most preferred on the left and least preferred on the right.

job	ranking	candidate	ranking
1	B, A, C	A	3, 2, 1
2	A, B, C	B	1, 2, 3
3	B, A, C	C	2, 1, 3

Given a matching, we say that an unpaired job and candidate form a *rogue couple* if both prefer each other to the partner they are currently matched to.

1. Select all of the rogue couples for the matching (1A, 2B, 3C) with respect to the preferences in the table above.

Review the definition of a matching and a rogue couple above. For more details read course note 4. Remember that a rogue couple requires the job and the candidate to *mutually* prefer each other!

- (a) (1, B)
- (b) (1, C)
- (c) (2, A)
- (d) (2, C)
- (e) (3, A)
- (f) (3, B)

Let's consider the choices in turn.

- a) Job 1 ranks candidate *B* above 1's current partner *A*, and candidate *B* likewise ranks job 1 above *B*'s current partner 2, so (1, *B*) *do* form a rogue couple.
- b) Candidate *C* prefers job 1 over *C*'s current partner 3, but job 1 prefers their current partner *A* over *C*, so (1, *C*) *do not* form a rogue couple.
- c) Job 2 prefers *A* over 2's current partner *B*, and candidate *A* prefers 2 over *A*'s current partner 1, so (2, *A*) *do* form a rogue couple.
- d) Candidate *C* prefers job 2 over *C*'s current partner 3, but job 2 prefers their current partner *B* over *C*, so (2, *C*) *do not* form a rogue couple.
- e) Job 3 prefers *A* over 3's current partner *C*, and candidate *A* prefers 3 over *A*'s current partner 1, so (3, *A*) *do* form a rogue couple.
- f) Job 3 prefers *B* over 3's current partner *C*, but candidate *B* prefers their current partner 2 over 3, so (3, *B*) *do not* form a rogue couple.

2. Again given the preferences above, select all of the rogue couples for the matching (3A, 2B, 1C).

Review the definition of a matching and a rogue couple above. For more details read course note 4. Remember that a rogue couple requires the job and the candidate to *mutually* prefer each other!

- (a) (1, A)
- (b) (1, B)
- (c) (2, A)
- (d) (2, C)
- (e) (3, B)
- (f) (3, C)

Let's consider each choice in turn.

- a) Job 1 prefers candidate *A* over 1's current partner *C*, but candidate *A* prefers their current partner 3 over 1, so (1, *A*) *do not* form a rogue couple.
- b) Job 1 and candidate *B* both prefer each over their current partners, so (1, *B*) *do* form a rogue couple.
- c) Job 2 prefers candidate *A* over 2's current partner *B*, but candidate *A* prefers their current partner 3 over 2, so (2, *A*) *do not* form a rogue couple.
- d) Candidate *C* prefers job 2 over *C*'s current partner 1, but job 2 prefers their current partner *B* over *C*, so (2, *C*) *do not* form a rogue couple.
- e) Job 3 prefers candidate *B* over 3's current partner *A*, but candidate *B* prefers their current partner 2 over 3, so (3, *B*) *do not* form a rogue couple.
- f) Job 3 prefers their current partner *A* over *C*, and candidate *C* prefers their current partner 1 over 3, so (3, *C*) *do not* form a rogue couple.

A matching is considered *stable* if it contains no rogue couples. This is because no one can convince anyone else to leave their partner in order to form a new pair. The propose-and-reject algorithm allows us to find stable matchings. This algorithm is described in course note 4. The following problems are designed to give you practice with this algorithm.

3. Consider the preference ranks given below for the jobs (1,2,3,4) and candidates (A,B,C,D).

job	ranking	candidates	ranking
1	A, B, D, C	A	3, 1, 4, 2
2	A, C, B, D	B	2, 1, 3, 4
3	B, C, D, A	C	4, 1, 3, 2
4	B, A, D, C	D	1, 2, 3, 4

Simulate the propose-and-reject algorithm (with jobs proposing), and answer the following questions about the algorithm. We consider the first offers to have taken place on day 1.

- (a) Which candidate did job 1 propose to on day 1?
1's top choice, *A*.
- (b) Was job 1 rejected?
 - Yes
 - No
No, candidate *A* received proposals from jobs 1 and 2, and *A* preferred 1 over 2.
- (c) Which candidate did job 4 propose to on day 1?
4's top choice, *B*.
- (d) Was job 4 rejected on day 1?
 - Yes
 - No
Yes, candidate *B* received proposals from jobs 3 and 4, and *B* preferred 3 over 4.
- (e) Which job proposed to candidate *A* on day 2, that did not propose to *A* on day 1?
Job 4, because 4 was rejected by candidate *B* on day 1, and *A* is the next candidate on 4 list.
- (f) Was job 4 rejected on day 2?
 - Yes
 - No
Yes, candidate *A* received proposals from jobs 4 and 1, and *A* preferred 1 over 4.
- (g) Which candidate was job 1 matched to when the algorithm terminated?
Candidate *A*.
- (h) Which candidate was job 2 matched to when the algorithm terminated?
Candidate *C*.
- (i) Which candidate was job 3 matched to when the algorithm terminated?
Candidate *B*.
- (j) Which candidate was job 4 matched to when the algorithm terminated?
Candidate *D*.
- (k) Now let us simulate the algorithm with candidates proposing. Before finding the outcome, what do you think is going to happen to the job's matches (compared to when the jobs were proposing)?
 - Each job will either get the same match as before, or prefer their new match.
 - Each job will either get the same match as before, or prefer their old match.
 - Some jobs will get the same match as before, some will prefer their new matches, and some will prefer their old matches.

The algorithm run with jobs proposing produces a job-optimal pairing. Therefore the original matches are as high on their list as each job can hope for. Another way to arrive at the same conclusion is to note that the algorithm produces job-pessimal pairings if run with candidates proposing. Therefore jobs get partners that are as low as it can be on their list in the second run.

- (l) Which job will candidate *A* propose to on the first day?
A's top choice, 3.
- (m) Which candidate gets rejected on the first day?
 - *A*
 - *B*
 - *C*
 - *D*
 - No one
No one gets rejected, because all jobs receive exactly one proposal.
- (n) In a job-optimal stable matching, which candidate would be paired with job 4?
The algorithm with jobs proposing returns a job-optimal pairing. We have already seen that the algorithm run with job proposing matches job 4 with candidate *D*.
- (o) In a candidate-optimal stable matching, which candidate would be paired with job 4?
The algorithm with candidates proposing returns a candidate-optimal pairing. In that matching, job 4 is matched with candidate *C*.
- (p) In a job-pessimal stable matching, which job would be paired with candidate *A*?
We get the job-pessimal pairing by running the algorithm with candidates proposing. In that case candidate *A* is matched with job 3.
- (q) In a candidate-pessimal stable matching, which job would be paired with candidate *A*?
We get the candidate-pessimal pairing by running the algorithm with jobs proposing. In that case candidate *A* is matched with job 1 as we have already seen.